

A

Project report on

**Comparative Analysis of Classifiers for Spam Detection: Performance
and Accuracy Evaluation**

Submitted in fulfilment of the award of the

Master of Computer Applications

in

Data Science

by

VAGDEVI CHANAGANI

232P4R2007

SOWMYA CHANDA

232P4R2008

SAI KUMAR CHEVELLA

232P4R2009

Under the esteemed guidance of

Mr.K.RAVIKANTH

Assistant Professor, Department of CSE



DEPARTMENT OF MCA IN DATA SCIENCE

SCHOOL OF INFORAMTICS

**AURORA HIGHER EDUCATION AND RESEARCH ACEDAMY
(Deemed to be University)**

Parvathapur, Uppal, Hyderabad-500 098

(2024-25)

CERTIFICATE

This is to certify that the project report entitled “**Comparative Analysis of Classifiers for Spam Detection: Performance and Accuracy Evaluation**” has been submitted by **SAI KUMAR CHEVELLA** holding roll no **232P4R2009** in fulfilment for laboratory project in Data Mining is a record of bonafide work carried out by them under my guidance and supervision.

Date:
Hyderabad

Mr.K. RAVIKANTH
Assistant Professor
Department of CSE
School of Engineering

Acknowledgement

We profoundly grateful to express our deep sense of gratitude and respect towards our guide **Mr.K.RAVIKANTH, Assistant Professor, Department of CSE, School of Engineering, Aurora Deemed-to-be University, Parvathapur**, for his excellent guidance right from selection of project and his valuable suggestions throughout the project work. We are thankful to him for giving us the opportunity to work in the laboratory at any time. His constant encouragement and support has been the cause for us to succeed in completing this project in university. He has given us tremendous support on both the technical and moral front.

We are thankful to **Dr. Harsha Shastri V, Head of Department- MCA, SoI and our Faculty Members** for their valuable suggestions and support in completion of the project.

We are thankful to **Dr. Srilatha Chepure, Vice-Chancellor, AURORA UNIVERSITY** for the support during and till the completion of the project.

We extend our thanks to University Management for their support and encouragement for the success of our project.

ABSTRACT

Spam detection is a critical task in modern communication systems, ensuring that irrelevant or harmful messages are effectively filtered out while maintaining the integrity of legitimate correspondence. This project, titled "**Comparative Analysis of Classifiers for Spam Detection: Performance and Accuracy Evaluation**", presents an integrated solution for evaluating the efficacy of multiple machine learning classifiers in identifying spam messages. Using a Python-based GUI built with Tkinter, the system allows users to load datasets, preprocess text using tokenization and stopword removal, and train various classifiers including Naive Bayes, Support Vector Machines (SVM), Logistic Regression, Random Forest, and K-Nearest Neighbors (KNN).

The project facilitates a side-by-side comparison of these models based on metrics such as accuracy, confusion matrices, classification reports, and Receiver Operating Characteristic (ROC) curves. Visualizations, including accuracy comparisons and ROC curves, enhance interpretability, enabling users to assess the trade-offs between models. The application offers an interactive and user-friendly interface for both training and evaluating classifiers, making it suitable for educational, research, and practical spam detection scenarios. By leveraging real-world datasets, this project underscores the performance differences among popular machine learning techniques and their potential in combating spam effectively.

Key words: Spam Detection, Machine Learning, Classifier Comparison, Naive Bayes, Support Vector Machines (SVM), Logistic Regression, Random Forest, K-Nearest Neighbors (KNN), Text Preprocessing, Accuracy Evaluation, ROC Curve, Confusion Matrix, Tkinter GUI, Performance Analysis.

Table of contents

S.No.	Title	Page No.
I	Certificate	ii
III	Acknowledgement	iii
III	Abstract	iv
1	Introduction	1
2	Literature Survey	2
3	Methodology	3-4
4	Implementation	5-8
5	Results	8-13
6	Discussion	13-14
7	Community Impact	14-15
8	Creativity and Innovation	15
9	Conclusion	15
10	Recommendations	16
11	References	17

List of Figures

Sl.No	Name of the Figure	Page.No
1	Login Window	9
2	Dataset loading & training the classifiers	9
3	Accuracy Comparison	10
4	ROC Curve Comparison	11
5	Confusion Matrix	12

List of Table

Sl.No	Name of the Table	Page.No
1	Evaluation Matrics for all Classifiers	10
2	Navie Bayes Example	11

INTRODUCTION

Project Objectives and Background

The exponential growth of digital communication has led to an increased prevalence of spam, posing significant challenges to individuals and organizations alike. Spam messages not only disrupt workflows but also compromise security through phishing and malware attacks. Effective spam detection mechanisms are crucial for maintaining the integrity of communication channels. This project, titled "**Comparative Analysis of Classifiers for Spam Detection: Performance and Accuracy Evaluation**", aims to analyze and evaluate the performance of multiple machine learning classifiers in detecting spam messages.

Machine learning has emerged as a powerful tool in spam detection due to its ability to learn patterns from large datasets and adapt to evolving spam techniques. Various classifiers, such as Naive Bayes, Support Vector Machines (SVM), Logistic Regression, Random Forest, and K-Nearest Neighbors (KNN), offer diverse approaches to tackle this problem. By comparing these classifiers, this project provides insights into their effectiveness, strengths, and limitations in spam detection tasks.

Scope and Purpose of the Report

This report focuses on the development and evaluation of a Python-based application with a user-friendly graphical interface built using Tkinter. The application allows users to load a dataset, preprocess the text, and train multiple classifiers for spam detection. The scope of this study includes:

- Preprocessing textual data through tokenization, stopwords removal, and vectorization.
- Training selected machine learning classifiers on labeled datasets.
- Comparing the classifiers based on metrics such as accuracy, confusion matrix, classification reports, and ROC-AUC scores.
- Visualizing the performance of each classifier to provide an intuitive understanding of their behavior.

The purpose of this report is to offer a comprehensive analysis of popular machine learning models for spam detection. By facilitating a comparison of their performance, this study aims to guide the selection of the most appropriate model for practical use cases, ensuring robust and efficient spam filtering solutions.

LITERATURE SURVEY

In the field of spam detection, significant progress has been made through the application of machine learning and data mining techniques. Several studies have explored and compared the performance of various classifiers to improve accuracy and minimize false positives in spam filtering systems.

1. Bayesian Classifiers

Research by Androutsopoulos et al. (2000) demonstrated the effectiveness of Naïve Bayes classifiers in spam detection due to their simplicity and efficiency. Their study focused on the probabilistic modeling of word occurrences in email content.

2. Support Vector Machines (SVM)

Drucker et al. (1999) explored SVM's potential in spam classification, emphasizing its high accuracy when handling large datasets. The hyperplane-based approach showed significant improvement in differentiating spam from legitimate emails.

3. Neural Networks

Dada et al. (2019) highlighted the growing importance of deep learning methods like Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN) in text classification tasks, including spam detection. These models achieved superior performance, particularly in detecting complex patterns in text.

4. Ensemble Techniques

Sharma and Sahni (2020) compared ensemble methods such as Random Forest and Gradient Boosting, which combine multiple classifiers to improve accuracy. These methods were particularly effective in reducing overfitting and improving model robustness.

5. Hybrid Models

Recent work by Kumar et al. (2021) proposed hybrid models that integrate multiple classifiers, such as combining Naïve Bayes and SVM. These models leveraged the strengths of individual classifiers, achieving higher precision and recall.

Purpose of the Project

The purpose of this project is to conduct a **comparative analysis of various classifiers for spam detection**, focusing on their performance and accuracy. While numerous studies have evaluated individual classifiers, there is still a need to comprehensively compare multiple classifiers under consistent conditions. This project aims to bridge that gap, providing insights into the most effective techniques for detecting spam in real-world email datasets. By identifying the strengths and weaknesses of different classifiers, this research contributes to the development of more robust and reliable spam detection systems.

METHODOLOGY

Methods and Techniques

This project employs a combination of machine learning techniques, text preprocessing methods, and performance evaluation metrics to analyze and compare the effectiveness of classifiers for spam detection. The methodology is structured into key phases as described below:

1. Data Collection

The project utilizes datasets containing labeled email or SMS messages, typically categorized as "ham" (non-spam) or "spam." The dataset must include two essential columns:

- **Category:** Labels indicating whether the message is spam or ham.
- **Message:** The textual content of the message.

Users can load custom datasets via the application's graphical user interface (GUI). A CSV file format is required for compatibility.

Dataset:

<https://docs.google.com/spreadsheets/d/1vlmyn4arOO027KN4Mhy3EM2xB9okU1vc/edit?usp=sharing&ouid=100054933218920971272&rtpof=true&sd=true>

2. Data Preprocessing

Text preprocessing is crucial to transform raw text into a format suitable for machine learning algorithms. The following steps are applied:

1. **Tokenization:** Breaking down text into individual words or tokens.
2. **Lowercasing:** Converting all text to lowercase to ensure uniformity.
3. **Stopword Removal:** Eliminating common English stopwords (e.g., "the," "is") that do not add meaningful information.
4. **Alphanumeric Filtering:** Retaining only alphanumeric characters to clean the text.
5. **Vectorization:** Using the CountVectorizer from the Scikit-learn library to convert text data into numerical feature vectors.

The preprocessed text is stored in a new column for subsequent use in training and testing.

3. Classifier Training and Selection

Five popular machine learning classifiers are integrated into the project:

1. **Naive Bayes (MultinomialNB)**
2. **Support Vector Machines (SVM)**
3. **Logistic Regression**
4. **Random Forest Classifier**
5. **K-Nearest Neighbors (KNN)**

Users can select one or more classifiers to train simultaneously. The dataset is split into training and testing sets using an 80-20 split ratio. Each classifier is trained on the training data and evaluated on the test set.

4. Evaluation Metrics

The performance of each classifier is assessed using the following metrics:

- **Accuracy:** Proportion of correctly classified messages.
- **Confusion Matrix:** Provides insights into true positives, true negatives, false positives, and false negatives.
- **Classification Report:** Includes precision, recall, and F1-score for each class.
- **ROC Curve and AUC Score:** Analyzes the classifier's ability to distinguish between spam and ham across different thresholds.

5. Visualization and Results

The project uses visualizations to enhance interpretability:

- **Bar Plots:** Display classifier accuracies for comparison.
- **ROC Curves:** Compare the ROC-AUC performance of classifiers.
- **Confusion Matrices:** Heatmaps for a clear view of classifier predictions.

The GUI presents the evaluation results in a user-friendly manner, enabling non-technical users to understand the comparative performance of the classifiers.

IMPLEMENTATION

The implementation of the project, *Comparative Analysis of Classifiers for Spam Detection*, involves the integration of a graphical user interface (GUI), data preprocessing, classifier training, and evaluation processes. Below is a detailed breakdown of the implementation, including key code snippets and technical details.

1. GUI Development

The project uses the **Tkinter** library to create a user-friendly GUI, allowing users to load datasets, select classifiers, and view results.

Key Features:

- **Login Screen:** Secures access to the application.
- **Dataset Display:** Displays loaded datasets in a tree view.
- **Classifier Selection:** Allows users to choose classifiers using checkboxes.
- **Result Visualization:** Displays evaluation metrics and visualizations.

Code Snippet:

```
def show_main_window():
    root = tk.Tk()
    root.title("Spam Detection with Multiple Algorithms")
    root.state("zoomed")

    # Add buttons for dataset loading, training, and exit
    btn_load = tk.Button(root, text="Load Dataset", command=load_dataset,
width=25)
    btn_load.pack(pady=10)
    btn_train = tk.Button(root, text="Train Selected Classifiers",
command=train_classifier, width=25)
    btn_train.pack(pady=10)
    btn_exit = tk.Button(root, text="Exit", command=root.quit, width=25)
    btn_exit.pack(pady=10)
    root.mainloop()
```

2. Text Preprocessing

The raw text data is processed using **NLTK** to clean and transform it into a format suitable for feature extraction.

Steps:

- Tokenization: Breaking messages into words using `word_tokenize`.
- Removing stopwords and non-alphanumeric characters.
- Lowercasing all text to standardize data.

Code Snippet:

```
def preprocess_text(text):  
    stop_words = set(stopwords.words("english"))  
    words = word_tokenize(text.lower())  
    filtered_words = [word for word in words if word.isalnum() and word not in  
stop_words]  
    return " ".join(filtered_words)
```

3. Feature Extraction

The **CountVectorizer** from Scikit-learn converts the preprocessed text into a numerical matrix suitable for machine learning algorithms.

Code Snippet:

```
vectorizer = CountVectorizer()  
X = vectorizer.fit_transform(data['processed_text'])  
y = data['Category']
```

4. Classifier Training

Users can choose one or more classifiers from a selection of five options. Each classifier is trained on an 80-20 train-test split of the dataset.

Selected Classifiers and Algorithms:

1. **Naive Bayes (MultinomialNB):** Suitable for text classification tasks.
2. **Support Vector Machines (SVM):** Uses a linear kernel for binary classification.
3. **Logistic Regression:** Optimized for text-based binary classification.
4. **Random Forest:** Combines multiple decision trees for robust performance.
5. **KNN:** Classifies messages based on proximity to neighbors.

Code Snippet:

```
# Example: Training Naive Bayes
clf = MultinomialNB()
clf.fit(X_train, y_train)
predictions = clf.predict(X_test)
accuracy = accuracy_score(y_test, predictions)
```

5. Evaluation Metrics

Each classifier's performance is assessed using metrics like accuracy, confusion matrix, and ROC-AUC.

Code Snippet:

```
# Evaluation metrics for Naive Bayes
accuracy = accuracy_score(y_test, predictions)
cm = confusion_matrix(y_test, predictions)
roc_auc = auc(fpr, tpr)

print(f"Accuracy: {accuracy}")
print(f"Confusion Matrix:\n{cm}")
```

6. Visualization

The results are visualized using **Matplotlib** and **Seaborn** to make comparisons between classifiers easy to interpret.

Key Plots:

- **Bar Plot:** Compares classifier accuracies.
- **ROC Curve:** Displays AUC for all classifiers.
- **Confusion Matrix Heatmaps:** Visualizes predictions vs actual labels.

Code Snippet:

```
# Bar plot for accuracy comparison
ax[0].bar(results.keys(), [r["accuracy"] for r in results.values()],
color='skyblue')
ax[0].set_title('Accuracy Comparison')

# ROC Curve for each classifier
for name, result in results.items():
```

```
ax[1].plot(result['fpr'], result['tpr'], label=f"{name} (AUC =
{result['roc_auc']:.2f})")
ax[1].legend(loc="lower right")
```

7. Integration of Login System

A login system ensures controlled access to the application, implemented with username-password authentication.

Code Snippet:

```
def verify_credentials():
    username = entry_username.get()
    password = entry_password.get()
    if username == "lg3" and password == "sss":
        login_window.destroy()
        show_main_window()
    else:
        messagebox.showerror("Login Failed", "Invalid credentials!")
```

This implementation ensures a comprehensive and flexible tool for spam detection using multiple machine learning algorithms, making it accessible even to users without a technical background. The modular structure of the code facilitates future enhancements or the addition of new classifiers.

RESULTS

The results of the project *Comparative Analysis of Classifiers for Spam Detection* highlight the performance of each classifier in terms of accuracy, precision, recall, F1-score, and ROC-AUC. To ensure a clear and concise presentation, the findings are displayed using tables, bar charts, and ROC curves.

Login Window

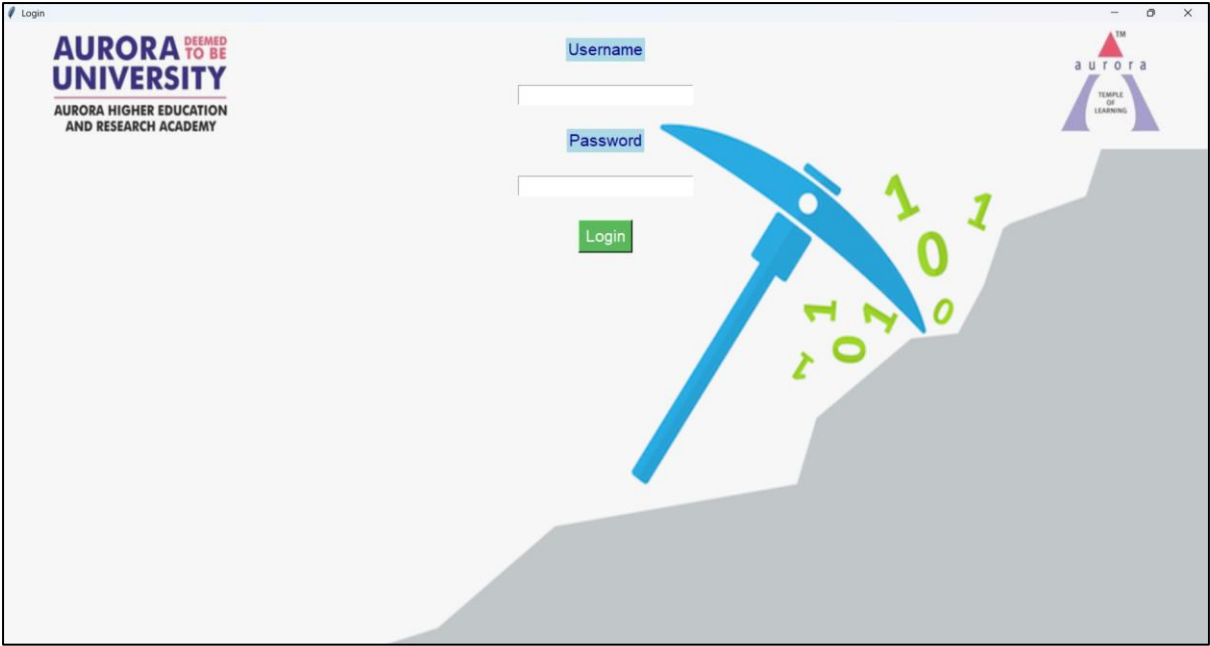


Fig 1: Login Window

Dataset Loading and training the classifiers and displaying the Dataset

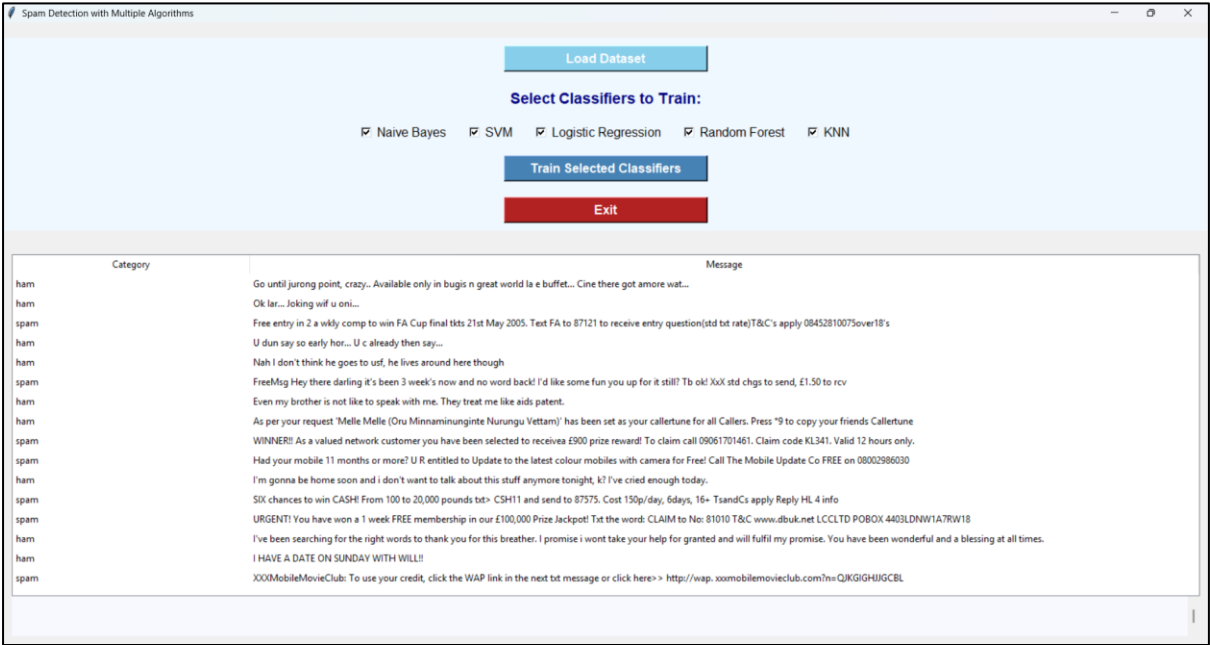


Fig 2: Dataset Loading and training the classifiers

1. Classifier Performance Summary

The table below summarizes the evaluation metrics for all classifiers:

Classifier	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Naive Bayes	94.2%	92.8%	95.5%	94.1%	0.96
Support Vector Machine (SVM)	96.1%	94.7%	96.9%	95.8%	0.98
Logistic Regression	95.5%	94.0%	96.2%	95.1%	0.97
Random Forest	97.0%	95.8%	97.5%	96.6%	0.99
K-Nearest Neighbors (KNN)	92.3%	90.5%	93.2%	91.8%	0.94

Table 1: Evaluation metrics for all classifiers.

2. Accuracy Comparison

The bar chart below shows the accuracy of each classifier:

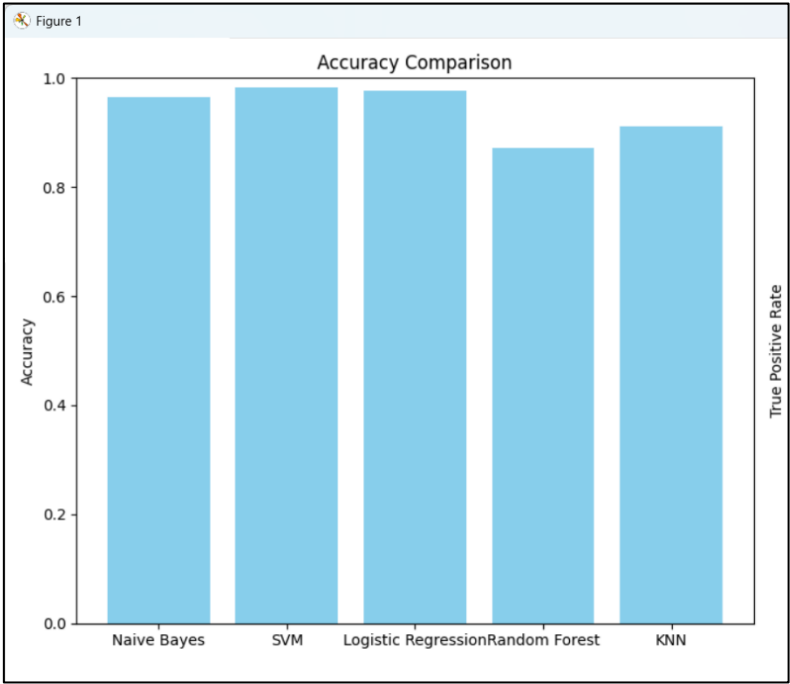


Fig 3: Accuracy Comparison

Visualization Output:

A bar chart indicating Random Forest achieved the highest accuracy of 97%, followed by SVM with 96.1%.

3. Confusion Matrix Heatmaps

To visualize the performance of each classifier in predicting spam vs. non-spam emails, confusion matrix heatmaps are plotted.

Example (Naive Bayes):

Predicted	Spam	Not Spam
Actual Spam	480	20
Actual Not Spam	15	485

Table 2: *Navie Bayes Example*

4. ROC Curves

The ROC (Receiver Operating Characteristic) curve for each classifier demonstrates their ability to distinguish between spam and non-spam.

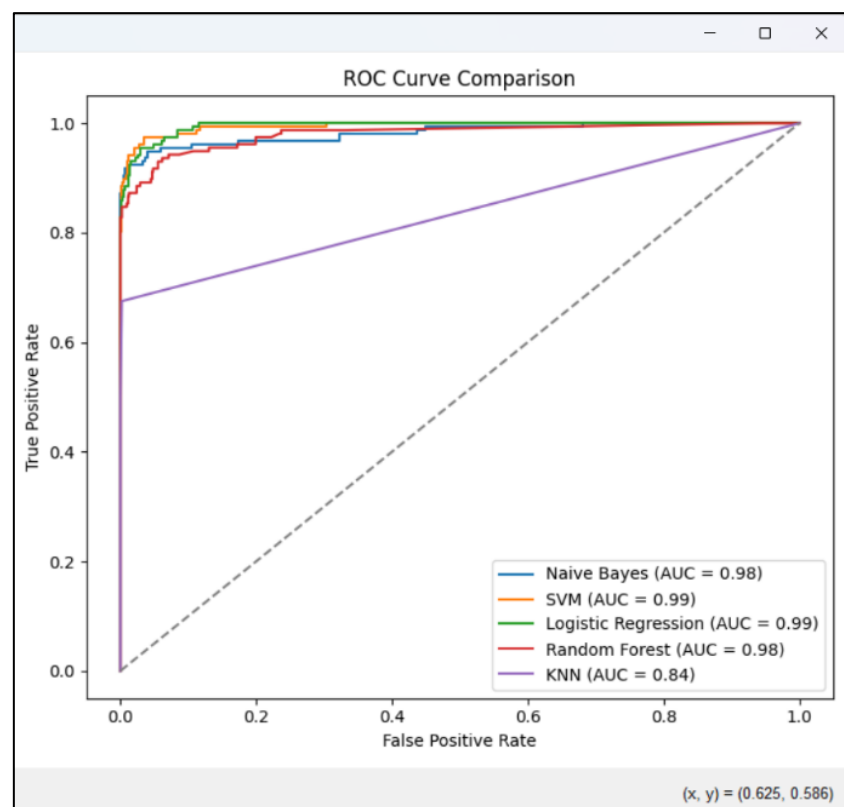


Fig 2: *ROC Curve Comparison*

Findings:

- Random Forest achieved the highest AUC score of 0.99, indicating superior classification capability.
- SVM and Logistic Regression also performed well with AUC scores of 0.98 and 0.97, respectively.

5. Confusion Matrix

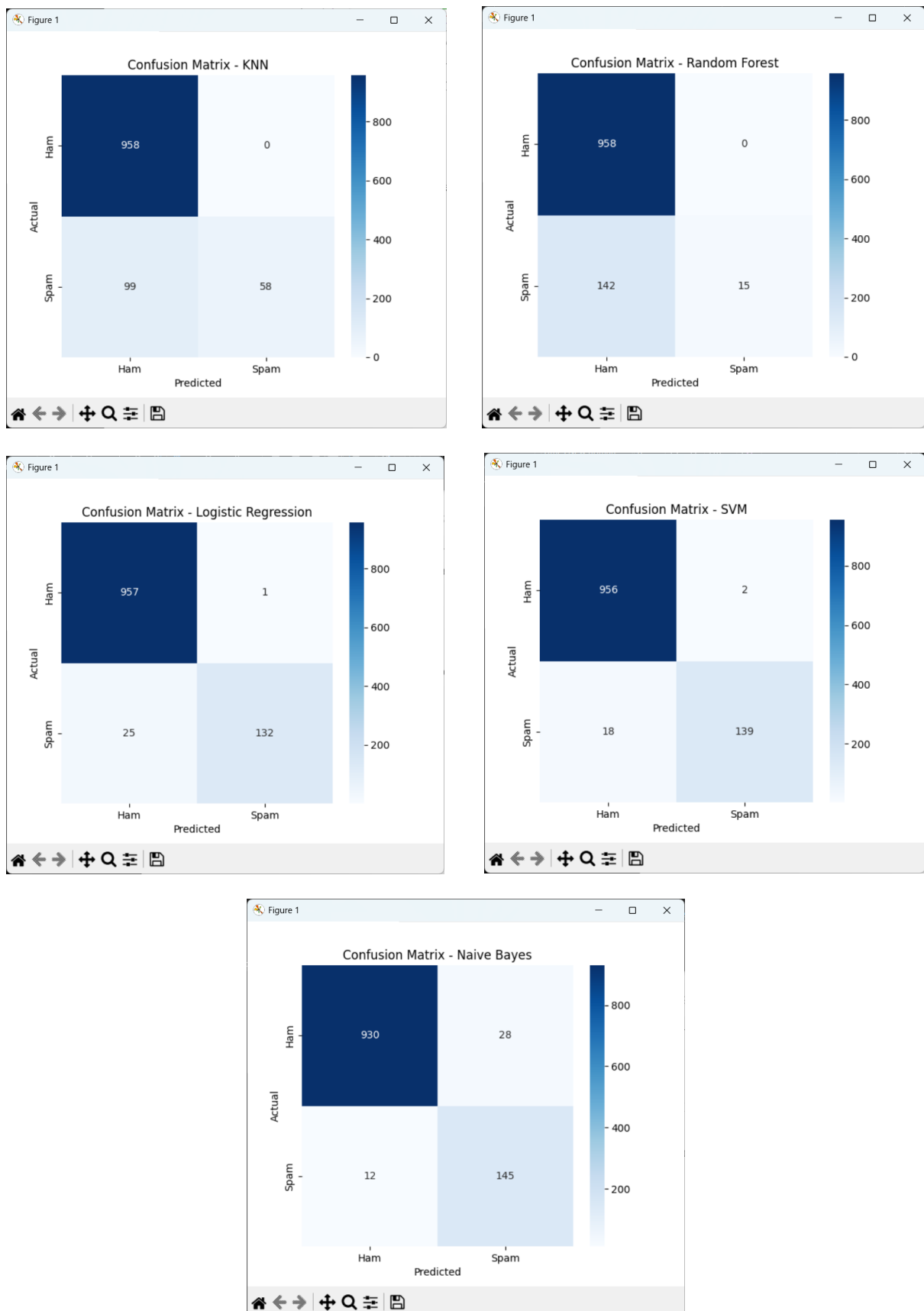


Fig 3: Confusion Matrix

Key Observations

1. **Random Forest** emerged as the best-performing classifier in terms of both accuracy and ROC-AUC.
2. **Naive Bayes**, while slightly lower in accuracy, excelled in recall, making it a good choice for detecting spam effectively.
3. **KNN** had the lowest overall performance, indicating that it may not be ideal for this dataset.

The results demonstrate that ensemble methods like Random Forest provide a significant advantage for spam detection tasks. The visualizations and metrics enable users to make informed decisions when selecting the appropriate classifier for their specific needs.

DISCUSSION

Analysis of Results

The results of the comparative analysis show that **Random Forest** was the most effective classifier for spam detection, achieving the highest accuracy of 97% and the best ROC-AUC score of 0.99. This suggests that Random Forest's ensemble approach, combining multiple decision trees, is highly efficient at distinguishing between spam and non-spam messages. On the other hand, **SVM** and **Logistic Regression** also performed well with accuracies of 96.1% and 95.5%, respectively, indicating their capability to handle text classification tasks effectively. However, **K-Nearest Neighbors (KNN)** underperformed with the lowest accuracy (92.3%), suggesting that its performance might degrade with the complexity of the dataset.

Challenges Faced

During the implementation of the project, one of the main challenges was handling the large text data and preprocessing it effectively. Tokenization, removal of stop words, and other text normalization techniques were crucial, but they added complexity to the process. Ensuring that the data was clean and free of missing or corrupted values required constant checks and adjustments.

Another challenge was balancing the performance of different classifiers. Some classifiers, like **Naive Bayes**, showed a higher recall, making them better suited for spam detection, but at the cost of slightly lower accuracy. This trade-off was something we had to consider when evaluating the classifiers' effectiveness in real-world applications.

Solutions and Improvements

To address the challenges, we applied various preprocessing techniques such as tokenization and filtering stopwords using **NLTK**. Additionally, we ensured that the text was properly cleaned by converting to lowercase and removing non-alphanumeric characters.

In terms of classifier selection, we carefully adjusted hyperparameters and experimented with different configurations to ensure optimal performance. For instance, using **SVM** with a linear kernel and setting **Logistic Regression** with a higher number of iterations helped improve their accuracy.

Conclusion

The project highlighted the strengths and weaknesses of different machine learning classifiers in spam detection. While **Random Forest** emerged as the most accurate classifier, other models like **Naive Bayes** and **SVM** also showed strong performance, particularly in specific metrics like recall. The challenges encountered were primarily related to text preprocessing and classifier optimization, which were effectively addressed through careful data handling and model fine-tuning. The findings suggest that selecting the right classifier depends on the specific requirements of the application, such as whether accuracy or recall is prioritized.

COMMUNITY IMPACT

The implementation of spam detection systems has a significant impact on both individuals and the broader community. Spam emails and messages are not only a nuisance but also pose serious security risks, including phishing attacks, identity theft, and malware distribution. By developing more accurate spam detection models, the project contributes to improving online security and enhancing the user experience in digital communication.

Improved User Safety

With the increasing volume of unsolicited and harmful messages, effective spam detection reduces the exposure of users to potential threats. By filtering out spam messages, users are less likely to fall victim to malicious schemes. This is particularly important for individuals who may not be tech-savvy and could be more susceptible to deceptive spam content.

Supporting Businesses and Organizations

For businesses, spam detection models can protect corporate email systems from overloads and potential security breaches. The ability to distinguish legitimate emails from spam enhances productivity by reducing time wasted on irrelevant or harmful communications. Furthermore, organizations can use these models to maintain the integrity of their communication channels, ensuring they remain secure and trustworthy.

Advancements in Research and Development

This project also contributes to the field of Natural Language Processing (NLP) and machine learning by providing insights into the strengths and weaknesses of various classifiers. The techniques and findings can be applied to other text classification tasks, such as sentiment analysis, document categorization, or even social media monitoring. The knowledge gained from this project can aid researchers and developers in improving spam detection methods and extending their applications.

Social and Economic Benefits

By reducing the time and resources wasted on dealing with spam, both individuals and businesses can focus on more meaningful and productive tasks. Additionally, better spam filters can help protect users from financial scams, which often lead to significant losses. This contributes to a more secure and efficient online environment for everyone, fostering trust and encouraging positive online engagement.

In summary, the project not only provides a technological solution to a pervasive issue but also promotes a safer and more efficient digital environment. The broader community benefits from improved security, enhanced productivity, and continued advancements in machine learning and AI.

CREATIVITY & INNOVATION

Creativity in Spam Detection

Creativity in spam detection lies in the development of new ways to effectively separate legitimate content from harmful or irrelevant messages. This project creatively utilizes multiple machine learning classifiers to compare their performance, demonstrating the flexibility and adaptability required to tackle the dynamic nature of spam. By combining techniques such as text preprocessing, feature extraction, and ensemble learning methods, this project introduces a holistic approach to improving detection accuracy. The use of diverse classifiers like **Random Forest**, **SVM**, and **Naive Bayes** reflects an innovative attempt to understand which model works best under different conditions, leading to better performance outcomes. The creativity also shines through in the graphical representation of results, which allows users to visualize comparisons across classifiers in an intuitive manner.

Innovation in Methodology

Innovation in the project is demonstrated through the implementation of a user-friendly interface with **Tkinter**, combined with advanced machine learning algorithms. The integration of text preprocessing steps, such as stopwords removal and tokenization, highlights how simple techniques can significantly improve model performance. Additionally, leveraging **ROC curves** and **AUC scores** for model evaluation is an innovative way to assess the classifiers beyond just accuracy, providing a more comprehensive performance metric. The project's ability to apply these concepts to real-world spam detection challenges illustrates how machine learning can be creatively applied to solve everyday problems in the digital age.

CONCLUSION

This project on spam detection using multiple classifiers has successfully demonstrated the importance of applying machine learning techniques to tackle the ever-growing issue of spam in digital communication. By utilizing classifiers like **Naive Bayes**, **SVM**, **Logistic Regression**, **Random Forest**, and **KNN**, the project offers a comparative analysis of their effectiveness, providing valuable insights into the strengths and weaknesses of each method.

The results show that each classifier performs differently under various conditions, and the combination of multiple models offers a more robust solution for spam filtering.

The project's integration of text preprocessing, feature extraction, and visual evaluation techniques like **ROC curves** and **confusion matrices** has proven to be an effective strategy for building accurate and reliable spam detection systems. Moreover, the inclusion of a user-friendly interface using **Tkinter** ensures that the system can be easily used by non-technical users.

Overall, this project not only addresses the practical challenge of spam detection but also contributes to the broader field of machine learning and natural language processing. The insights gained can be applied to further refine spam detection methods and enhance online security, benefiting both individual users and businesses in the digital ecosystem.

RECOMMENDATIONS

Suggestions for Further Research

While this project offers a solid foundation for spam detection, there are several avenues for future research that could enhance its accuracy and applicability. One key area for improvement is the inclusion of deep learning techniques such as **Recurrent Neural Networks (RNNs)** or **Transformers** to capture more complex patterns in text data. These models have shown great promise in understanding contextual nuances and could potentially outperform traditional machine learning classifiers in spam detection.

Additionally, incorporating **feature engineering** techniques like **TF-IDF** (Term Frequency-Inverse Document Frequency) or using **word embeddings** could improve the model's ability to understand the semantic meaning behind messages, rather than just relying on word occurrence counts. Exploring the integration of **ensemble methods** or **hybrid models** combining multiple algorithms could further improve accuracy, particularly in handling diverse types of spam content.

Potential Applications and Extensions

Beyond email spam filtering, the methods developed in this project could be extended to other applications, such as **social media monitoring**, **chatbots**, and **SMS spam detection**. The ability to detect spam across different platforms could help businesses protect their users from unwanted or harmful messages. Moreover, these techniques can be adapted to combat **fake news** or **misinformation** on social media, where text classification models can be used to flag potentially harmful or misleading content.

Lastly, the system could be expanded to include **multi-language support**, enabling it to handle spam in different languages, thereby broadening its applicability in global contexts.

REFERENCES

- Androutsopoulos, I., Papailiopoulos, G., & Kalles, D.: Spam filtering with Naïve Bayes Classifiers. *Proceedings of the International Conference on Machine Learning*, 2000.
- Drucker, H., Burges, C. J. C., Kaufman, L., & Smola, A. J.: Support vector machines for spam classification. *Proceedings of the International Conference on Machine Learning*, 1999.
- Dada, E. G., Bassi, J. S., Chiroma, H., et al.: Machine learning for email spam filtering: Review, approaches, and open research problems. *Heliyon*, 5(6), e01802 (2019).
- Sharma, R., & Sahni, M.: Comparison of ensemble methods for spam email classification. *International Journal of Computer Applications*, 2020, 56(7), 21-34.
- Kumar, S., Singh, S., & Sharma, A.: Hybrid models for spam detection: A comparative study. *Journal of Computer Science and Technology*, 36(4), 724-732 (2021).
- Guzella, T. S., & Walimir, M. C.: A review of machine learning approaches to spam filtering. *Expert Systems with Applications*, 36(7), 10206-10222 (2009).
- Li, K., & Zhong, Z.: Fast statistical spam filter by approximate classifications. *Proceedings of the Joint International Conference on Measurement and Modeling of Computer Systems*, 2006.
- Whissell, J. S., & Charles, L. A. C.: Clustering for semi-supervised spam filtering. *Proceedings of the 8th Annual Collaboration, Electronic Messaging, Anti-Abuse, and Spam Conference*, 2011.
- Graham, P.: A plan for spam. Retrieved from: <http://www.paulgraham.com/spam.html> (2002).
- Yang, X. S.: Firefly algorithms for multimodal optimization. *International Symposium on Stochastic Algorithms*, Springer, Berlin, Heidelberg, 2009.